

Assignment 2

Black-Box Testing

Objective

The purpose of this assignment is to apply black-box testing techniques on a real codebase.

Black-Box Testing

Task Description

Select any existing personal or business application of your choice — it can be an application you use daily, a well-known software product, or any real-world system. From that application, identify a specific module or function that accepts inputs with defined valid ranges.

In your report, include a brief description of the application and the specific module chosen. Then select any two input variables from that module and apply the following black-box testing techniques on those two variables:

Part A — Boundary Value Analysis (BVA)

For each variable, identify its valid range. Test values at and just inside the boundaries. Calculate the number of required test cases using the appropriate formula, then write all test cases.

Part B — Robustness Testing

Extend your BVA test cases by also testing invalid values — just below the minimum and just above the maximum. Calculate the required number of test cases and write them all.

Part C — Worst-Case Testing

Apply worst case testing on the same module. Calculate the required number of test cases and write them all.

Part D — Robust Worst-Case Testing

Combine robustness and worst-case: test all combinations. Calculate the required number of test cases and write them all.

Part E — Equivalence Class Partitioning (Unit Level)

Select any one function or method from your chosen module. Identify the input domain and divide it into equivalence classes — both valid and invalid.

- Identify all valid equivalence classes

- Identify all invalid equivalence classes
- Write one test case per class

Test Case #	Input Field	Equivalence Class	Class Type	Input Value	Expected Output
TC-E Q01	(field name)	(describe class)	Valid	(value)	(output)
...	...	...	...	...	...

Part F — Decision Table Testing (Unit Level)

Identify a function in your application where the output depends on a combination of conditions (e.g., login validation, discount logic, form submission rules).

- List all conditions (C1, C2, ...) and possible actions/outputs (A1, A2, ...)
- Build a decision table
- Write one test case per rule

Structure your decision table as follows:

	R1	R2	R3	R4	R5
C1: (condition name)	T	T	F	F	T
C2: (condition name)	-	F	T	F	F
C3: (condition name)	-	-	T	F	T
Actions					
A1: (action/output)	X				X
A2: (action/output)		X	X		
A3: (action/output)				X	

Then write a test case for each rule.

Part G — Pairwise Testing

Select a feature in your application that has three or more input parameters, each with multiple possible values (e.g., a search form, a filter screen, a registration flow).

- List all input parameters (P1, P2, P3 ...) and their possible values
- Calculate the total number of all-combination test cases
- Now generate pairwise test cases
- State total all-combination count vs. pairwise count

Write your pairwise test cases in the following format:

Test Case ID	Input (P1)	Input (P2)	Input (P3)	Expected Output
1	(value)	(value)	(value)	
...	...	...	...	...

Submission Guidelines:

A single report (PDF or Word) containing:

- Brief description of the application and the chosen module
- The two input variables selected with their valid ranges
- Solutions for Parts A, B, C, D, E, F and G
- Conclusion